

FICHE DE TD 1/3 • FACILE

Informatique • Tle C D E

Programmation en langage C

Module 3 : Algorithmique et programmation

Ce TD porte sur les bases de la syntaxe C : structure, entrées/sorties, structures de contrôle, tableaux, enregistrements et recherche séquentielle.

EXERCICE 1 Structure d'un programme et vocabulaire

1. Complétez les directives de préprocesseur pour inclure les bibliothèques d'entrée/-sortie standard et des fonctions mathématiques :

```
#include _____  
#include _____
```

2. Vrai ou Faux? Justifiez brièvement.

- (a) La fonction `main` est le point d'entrée obligatoire de tout programme C.
- (b) Les instructions en C se terminent obligatoirement par un point-virgule ;.
- (c) Une variable déclarée `const float PI = 3.14;` peut voir sa valeur modifiée dans le programme.

3. Quelle est la traduction en C de l'instruction algorithmique Écrire("Résultat : ", x) si x est un entier?

EXERCICE 2 Entrées/Sorties : `scanf` et `printf`

1. Donnez l'instruction `printf` exacte pour afficher le message Entrez votre age : suivi d'un retour à la ligne.
2. Soit la déclaration `int a; float b;`. Écrivez l'unique instruction `scanf` permettant de lire une valeur entière pour a et une valeur réelle pour b (séparées par un espace à la saisie).
3. Complétez le code ci-dessous pour afficher la valeur de `rayon` (float) avec 2 chiffres après la virgule et la valeur de `volume` (float) en notation scientifique :

```
printf("Rayon = _____ ; Volume = _____\n", rayon, volume);
```

EXERCICE 3 Structures de contrôle conditionnelles et itératives

1. Traduisez en C la structure algorithmique suivante (indentation soignée) :

```
Si (note >= 10) Alors
    Écrire("Admis")
Sinon
    Écrire("Ajourné")
Fin Si
```

2. Écrivez la boucle `for` en C équivalente à : Pour `i` de 1 à 10 faire ... Fin Pour.
3. Complétez la boucle `while` ci-dessous pour qu'elle calcule la somme des entiers de 1 à 100 (inclus) dans la variable `somme` :

```
int i = 1, somme = 0;
while (_____) {
    somme = somme + i;
    _____;
}
```

4. Quelle est la différence fondamentale entre une boucle `while` et une boucle `do...while`?

EXERCICE 4 Tableaux : déclaration et parcours

1. Déclarez un tableau `notes` pouvant contenir 30 notes entières.
2. Écrivez la boucle `for` qui initialise toutes les cases de ce tableau à 0.
3. Soit `int T[5] = {10, 20, 30, 40, 50};`. Quelle est la valeur de `T[2]` ? De `T[5]` (expliquez) ?
4. Écrivez le fragment de code C qui lit au clavier 5 entiers pour les stocker dans un tableau `Tab[5]`.

EXERCICE 5 Enregistrements : struct

1. Définissez un type `Etudiant` (structure) contenant : un entier `matricule`, une chaîne de 30 caractères `nom`, et un réel `moyenne`.

2. Déclarez une variable `E1` de ce type et affectez-lui les valeurs : matricule 12345, nom "Dupont", moyenne 14.5 (utilisez la notation point).
3. Écrivez l'instruction `printf` pour afficher le nom et la moyenne de `E1`.
4. Comment accède-t-on au champ `moyenne` via un pointeur `Etudiant *pE` sur cette structure?

EXERCICE 6 Recherche séquentielle dans un tableau

On souhaite traduire en C l'algorithme de recherche séquentielle d'une valeur `cle` dans un tableau `T` de taille `N`.

1. Déclarez les variables nécessaires : le tableau `T` (max 100 entiers), `N` (taille effective), `cle`, un indice `i` et un booléen `trouve` (utilisez `int` pour le booléen).
2. Écrivez la boucle `for` ou `while` qui parcourt le tableau tant que `i < N` et que `trouve` est faux.
3. À l'intérieur de la boucle, écrivez le test `if` qui met `trouve` à vrai (1) si `T[i] == cle`.
4. Après la boucle, écrivez la structure `if/else` qui affiche "Trouvé à l'indice `i`" ou "Non trouvé".